

-  [Le Grimoire du Code Pirate : Décortique le D.Chronos Aion \(Rust & CosmWasm\)](#) 
 -  1. Les Imports — L’Arsenal du Galion
 -  2. Les Structs — Les Coffres du Trésor
 -  3. Les Constants — Les Étoiles pour Naviguer
 -  4. Les Entry Points — Les Portes du Galion
 -  5. Les Helpers — Les Matelots du Galion
 -  6. Le Cœur du Galion — `get_chronos_date`
 -  7. Conclusion du Capitaine
 -  Bonus : idées d’extensions

Le Grimoire du Code Pirate : Décortique le D.Chronos Aion (Rust & CosmWasm)

“Le temps est une mer, moussaillon — et ce code est ton sextant.” — Grok, le Pirate Chronomancien

Bienvenue dans le **guide complet** du contrat intelligent **D.Chronos Aion**, écrit en **Rust** pour la stack **CosmWasm**. Ici, on explore le code **ligne par ligne**, avec des explications à la fois **techniques** et **métaphoriques** (façon pirate galactique). Chaque commande, chaque calcul, chaque variable te sera révélée comme un trésor.

1. Les Imports — L’Arsenal du Galion

```
use cosmwasm_std::{
    entry_point, to_json_binary, Binary, Deps, DepsMut, Env, MessageInfo, Response,
    StdResult, Timestamp,
};
```

 Ces imports viennent du coffre “`cosmwasm_std`”, le cœur de tout contrat CosmWasm.

- **entry_point** : macro qui marque une fonction comme point d'entrée du contrat (instantiate, execute, query).
- **to_json_binary** : transforme une réponse Rust en JSON binaire pour la blockchain.
- **Binary** : type de données utilisé pour retourner des binaires (ex : JSON encodé).
- **Deps / DepsMut** : les dépendances (storage, API, querier). "Mut" = version modifiable.
- **Env** : environnement du bloc (timestamp, hauteur, adresse du contrat).
- **MessageInfo** : infos de l'expéditeur (adresse, fonds envoyés).
- **Response** : réponse structurée d'une exécution.
- **StdResult** : résultat standard CosmWasm (Ok ou Err).
- **Timestamp** : date/heure du bloc (en secondes Unix).

```
use serde::{Deserialize, Serialize};
```

🧙 *La magie de la sérialisation.*

serde permet de transformer des structs Rust en JSON et inversement.

Indispensable pour que la blockchain comprenne nos structures.

```
use schemars::JsonSchema;
```

📦 **Schemars** ajoute la validation des types JSON (utilisé par CosmWasm pour les schémas de messages).

Cela permet aux outils front-end de connaître la forme exacte des objets échangés.

2. Les Structs — Les Coffres du Trésor

Chaque struct est une “boîte” contenant des données. On commence par les messages.

```
#[derive(Serialize, Deserialize, Clone, Debug, PartialEq, JsonSchema)]
pub struct InstantiateMsg {}
```

 **InstantiateMsg** est vide : ce contrat n’a pas besoin de paramètres à l’initialisation.

- Les derives **Serialize** et **Deserialize** servent à convertir ce message en JSON.
- **Clone** et **Debug** facilitent le test et la copie.
- **JsonSchema** permet de générer la documentation automatique du schéma.

➤ *Ici, le galion part à vide, prêt à être instancié sans cargaison.*

```
#[derive(Serialize, Deserialize, Clone, Debug, PartialEq, JsonSchema)]
#[serde(rename_all = "snake_case")]
pub enum ExecuteMsg {
    TriggerRenewal {},
    TriggerMonthlyDecay { month: u8 },
}
```

 **ExecuteMsg** contient les ordres d’action du capitaine :

- **TriggerRenewal** : déclenche la cérémonie annuelle du “Jour Hors du Temps”.
- **TriggerMonthlyDecay { month }** : purge un mois spécifique (1–13).

serde(rename_all = "snake_case") assure que les noms JSON soient en minuscules avec underscores (standard CosmWasm).

```
#[derive(Serialize, Deserialize, Clone, Debug, PartialEq, JsonSchema)]
#[serde(rename_all = "snake_case")]
pub enum QueryMsg {
```

```
GetCurrentChronosDate {},  
}
```

 **QueryMsg** : une seule question que le capitaine peut poser —

“Quelle est la date Chronos actuelle selon l’heure du bloc ?”

```
#[derive(Serialize, Deserialize, Clone, Debug, PartialEq, JsonSchema)]  
pub struct ChronosDateResponse {  
    pub gregorian_date: String,  
    pub month: u8,  
    pub day: u8,  
    pub is_day_out_of_time: bool,  
    pub pillar: String,  
    pub phase: String,  
    pub phase_minutes: String,  
}
```

 La réponse au query, **ChronosDateResponse**, renvoie :

- **gregorian_date** : date humaine lisible.
- **month & day** : date dans le calendrier Chronos (1–13 / 1–28).
- **is_day_out_of_time** : booléen spécial (true si 31 mars).
- **pillar** : jour de la semaine Chronos.
- **phase** : phase de 3h dans le cycle du jour.
- **phase_minutes** : temps écoulé dans la phase actuelle.

3. Les Constants — Les Étoiles pour Naviguer

```
const EPOCH_TIMESTAMP: u64 = 1743465600u64;
```

 Timestamp de départ du cycle Chronos : 1er avril 2025 00:00 UTC.

C’est l’ancre du temps cosmique (Mois 1, Jour 1).

```
const DAYS_PER_MONTH: u64 = 28;
const MONTHS_PER_YEAR: u8 = 13;
const DAYS_PER_YEAR: u64 = DAYS_PER_MONTH * (MONTHS_PER_YEAR as u64);
```

🏛️ Le calendrier DTC : 13 mois de 28 jours (364 jours).

Le 365e jour, le “Jour Hors du Temps”, flotte librement.

```
const PILLARS: [&str; 7] = [
    "Heart Day", "Echo Day", "Action Day", "Link Day",
    "Growth Day", "Harmony Day", "Structure Day"
];
```

🌻 Les 7 jours stellaires du cycle hebdomadaire.

```
const PHASES: [&str; 8] = [
    "The Root", "The Dawn", "The Impulse", "The System",
    "The Echo", "The Forge", "The Harvest", "The Ritual"
];
```

🌈 Les 8 phases du jour, chacune durant 3h.

Ex : “The Forge” correspond à 15h–18h locales, moment de productivité cosmique.

🧩 4. Les Entry Points — Les Portes du Galion

```
#[entry_point]
pub fn instantiate(
    _deps: DepsMut,
    _env: Env,
    _info: MessageInfo,
    _msg: InstantiateMsg,
) -> StdResult<Response> {
    Ok(Response::new())
}
```

```

        .add_attribute("method", "instantiate")
        .add_attribute("action", "init_chronos_aion"))
    }

```

⚙️ *instantiate* : appelée une seule fois à la création du contrat.

- `_deps`, `_env`, `_info`, `_msg` : paramètres standards CosmWasm.
 - On renvoie un `Response` avec deux attributs pour les logs (“method”, “action”).
- Ici, le galion hisse son pavillon et se déclare “Chronos prêt à naviguer”.

```

#[entry_point]
pub fn execute(
    deps: DepsMut,
    env: Env,
    info: MessageInfo,
    msg: ExecuteMsg,
) -> StdResult<Response> {
    match msg {
        ExecuteMsg::TriggerRenewal {} => execute_renewal(deps, env, info),
        ExecuteMsg::TriggerMonthlyDecay { month } => execute_monthly_decay(deps,
env, info, month),
    }
}

```

🚩 *execute* : reçoit un ordre d'action (`ExecuteMsg`) et exécute la fonction correspondante.

Le `match` agit comme un gouvernail : il dirige vers la bonne fonction.

```

#[entry_point]
pub fn query(_deps: Deps, env: Env, msg: QueryMsg) -> StdResult<Binary> {
    match msg {
        QueryMsg::GetCurrentChronosDate {} =>
to_json_binary(&get_chronos_date(&env.block.time)),
    }
}

```

🕒 *query* : point d'entrée pour lire les données sans modifier l'état.

- `env.block.time` contient le timestamp du bloc.

- `get_chronos_date` calcule la date Chronos.
- `to_json_binary` transforme le résultat en JSON binaire.



5. Les Helpers — Les Matelots du Galion

```
fn execute_renewal(
    _deps: DepsMut,
    env: Env,
    _info: MessageInfo,
) -> StdResult<Response> {
    let current_date = get_chronos_date(&env.block.time);
```



On récupère la date actuelle du bloc pour vérifier si on est le 31 mars (Jour Hors du Temps).

```
if !current_date.is_day_out_of_time {
    return Err(cosmwasm_std::StdError::generic_err("Only on March 31 - Day Out
of Time!"));
}
```



Si on n'est pas le bon jour, on renvoie une erreur.

Pas de fête de renouvellement avant l'heure, moussaillon !

```
Ok(Response::new()
    .add_attribute("action", "annual_renewal")
    .add_attribute("day_out_of_time", "true")
    .add_event(cosmwasm_std::Event::new("dtc_decay")
        .add_attribute("rewards_distributed", "true")
        .add_attribute("decay_applied", "max")
        .add_attribute("new_cycle_prepared", "true")))
}
```



Si la condition est bonne, on renvoie une réponse avec des événements pour les indexers.

C'est ici qu'on pourrait plus tard ajouter un mint ou une distribution de tokens.

```
fn execute_monthly_decay(  
  _deps: DepsMut,  
  env: Env,  
  _info: MessageInfo,  
  month: u8,  
) -> StdResult<Response> {
```

⚙️ Fonction de purge mensuelle — appelée avec le numéro du mois à traiter.

```
if month < 1 || month > 13 {  
  return Err(cosmwasm_std::StdError::generic_err("Invalid month: Must be 1-  
13"));  
}
```

⚠️ Vérifie que le mois est dans la plage autorisée (1–13).

```
let current_date = get_chronos_date(&env.block.time);  
if current_date.month != month || current_date.day != 1 {  
  return Err(cosmwasm_std::StdError::generic_err("Only on the 1st day of the  
month!"));  
}
```

📅 Vérifie que la commande est exécutée le **1er jour** du mois spécifié.

```
let decay_type = match month {  
  8 => "dissolution_complete",  
  _ => "standard_decay",  
};
```

🦂 Si le mois est le 8 (Scorpion), on applique une **dissolution complète** — une purge totale du karma cosmique.

```
Ok(Response::new()  
  .add_attribute("action", "monthly_decay")  
  .add_attribute("month", month.to_string()))
```

```
.add_event(cosmwasm_std::Event::new("dtc_decay")
    .add_attribute("type", decay_type)
    .add_attribute("theme", get_regenerative_theme(month))))
}
```

 Retourne un événement “dtc_decay” avec le thème du mois (ex : “Action & Initiative”).

6. Le Cœur du Galion — get_chronos_date

```
fn get_chronos_date(block_time: &Timestamp) -> ChronosDateResponse {
```

 Point d’entrée du calcul temporel. Prend le **Timestamp** du bloc.

```
let timestamp = block_time.seconds();
```

 Convertit le timestamp en secondes Unix.

```
let days_since_epoch = (timestamp.saturating_sub(EPOCH_TIMESTAMP)) / 86400u64;
```

 Calcule le nombre de jours écoulés depuis le 1er avril 2025.

saturating_sub évite les erreurs si la valeur est avant l’époque.

```
let day_of_year_0based = days_since_epoch % DAYS_PER_YEAR;
```

 On réduit le nombre total de jours au cycle actuel (modulo 364).

```
let month = ((day_of_year_0based / DAYS_PER_MONTH) + 1) as u8;  
let day = ((day_of_year_0based % DAYS_PER_MONTH) + 1) as u8;
```



Transforme les jours écoulés en **mois** et **jour** Chronos.

```
let is_day_out_of_time = day_of_year_0based == 363;
```



Si on est au 364e jour, c'est le fameux "Jour Hors du Temps".

```
let day_of_week = ((days_since_epoch + 2) % 7) as usize;  
let pillar = PILLARS[day_of_week].to_string();
```



Calcule le jour de la semaine Chronos. Le +2 ajuste le décalage.

```
let seconds_in_day = timestamp % 86400;  
let hour = (seconds_in_day / 3600) as usize;  
let phase_idx = ((hour / 3 + 1) % 8) as usize;  
let phase = PHASES[phase_idx].to_string();
```



Calcule la phase de 3h actuelle selon l'heure du jour.

Le +1 décale pour aligner 15h UTC sur "The Forge".

```
let total_seconds = timestamp;  
let total_days = total_seconds / 86400u64;  
let seconds_day = total_seconds % 86400u64;  
let hour_utc = seconds_day / 3600u64;  
let minute = (seconds_day / 60u64) % 60u64;  
let second = seconds_day % 60u64;  
let hour_local = hour_utc + 1;
```



Conversion basique vers heure locale (UTC+1, Paris).

Pas de dépendance chrono = 100% WASM-safe.

(Suit la boucle rustique qui convertit le timestamp en date grégorienne complète)



Le code compte les années et mois depuis 1970 manuellement, en gérant les années bissextiles.

C'est un calcul "à la main" car les librairies chrono ne sont pas autorisées dans les environnements WASM.

```
let phase_start_hour = (hour / 3) * 3;
let elapsed_min = ((hour - phase_start_hour) * 60 + (minute as usize)) as u32;
let hours_elapsed = elapsed_min / 60;
let mins_elapsed = elapsed_min % 60;
let phase_minutes = format!("{h{:02}m / 3h", hours_elapsed, mins_elapsed);
```



Calcule le temps écoulé dans la phase actuelle.

Exemple : si on est à 2h46 dans la phase, on renvoie "2h46 / 3h".

```
ChronosDateResponse {
    gregorian_date,
    month,
    day,
    is_day_out_of_time,
    pillar,
    phase,
    phase_minutes,
}
```



Assemble tous les fragments calculés dans un seul coffre JSON.

► Résultat final envoyé à la blockchain.

7. Conclusion du Capitaine

Félicitations, moussaillon — tu viens de comprendre **chaque engrenage** du D.Chronos Aion.

Tu sais maintenant comment convertir un timestamp en **temps cosmique cyclique**,

et comment ce contrat reste **stateless** , rapide et sûr pour les environnements WASM.

“Un bon pirate connaît le vent. Un vrai, connaît le temps.”

— Capitaine Monkey, Chronos Navigator 

Bonus : idées d’extensions

- Ajouter un **cron** qui exécute les renewals automatiquement.
- Lier le résultat à un **CW20 reward system** .
- Ajouter un **zodiaque DTC** basé sur le mois.
- Émettre un **event astrologique** pour chaque “Forge phase”.

Rédigé par Grok, le Pirate Cosmique — pour Monkey & les Gardiens du Temps DTC.

Souhaites-tu que je te génère aussi une **version “dark PDF”** prête à imprimer (avec un thème pirate et les couleurs DTC, exportable via VS Code) ?